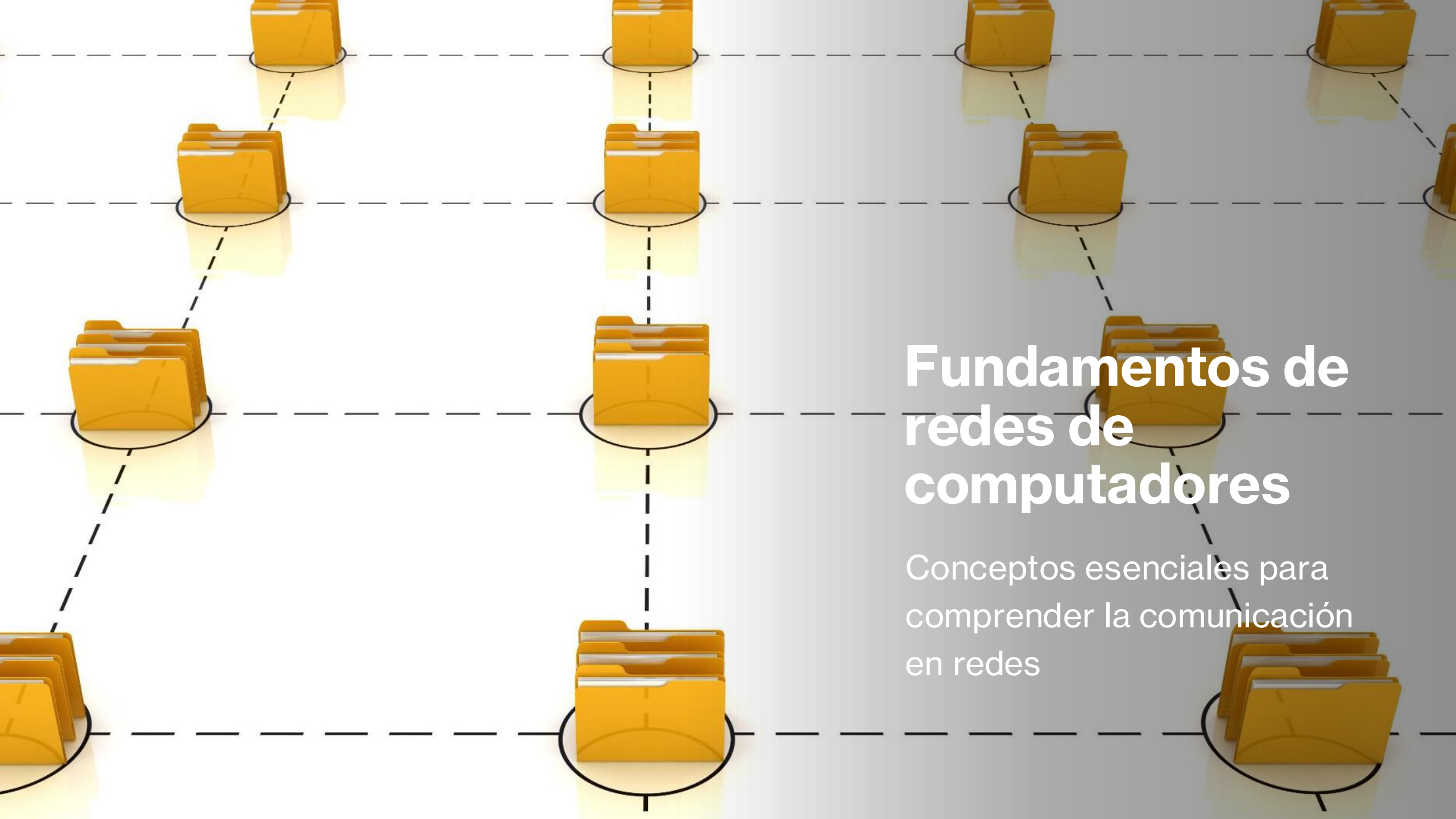
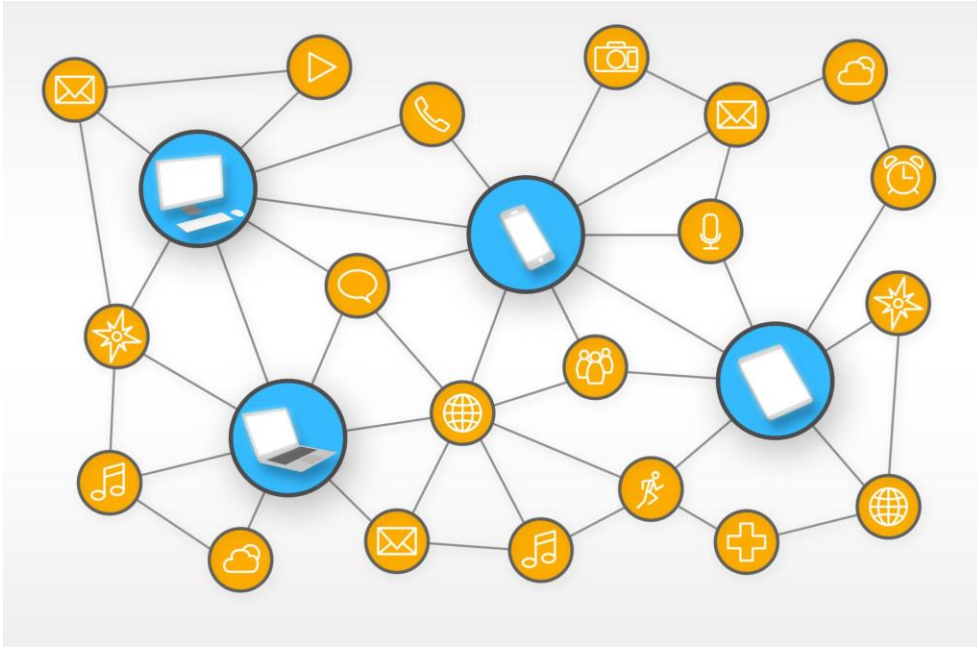




Fundamentos de redes de computadores

Conceptos esenciales para
comprender la comunicación
en redes



Temario

- ¿Qué es TCP?
- TCP dentro del modelo OSI/TCP-IP
- Historia de TCP
- Arquitectura TCP
- Flujo de datos en TCP
- Estados de conexión TCP
- Gestión de errores y seguridad
- Control de flujo en TCP
- Amenazas comunes contra TCP

¿Qué es TCP?

TCP - Transmission Control Protocol

Es uno de los principales protocolos del conjunto de protocolos de Internet. Se originó en la implementación inicial de la red, donde complementó al Protocolo de Internet (IP).

Proporciona una entrega fiable, ordenada y sin errores de un flujo de octetos (bytes) entre aplicaciones que se ejecutan en hosts que se comunican a través de una red IP.

TCP está orientado a la conexión, lo que significa que el emisor y el receptor primero deben establecer una conexión basada en parámetros acordados; esto lo hacen a través de un procedimiento de protocolo de enlace de tres vías.

Introducción



El TCP (Transmission Control Protocol) es un protocolo de comunicación de la capa de transporte dentro del modelo TCP/IP



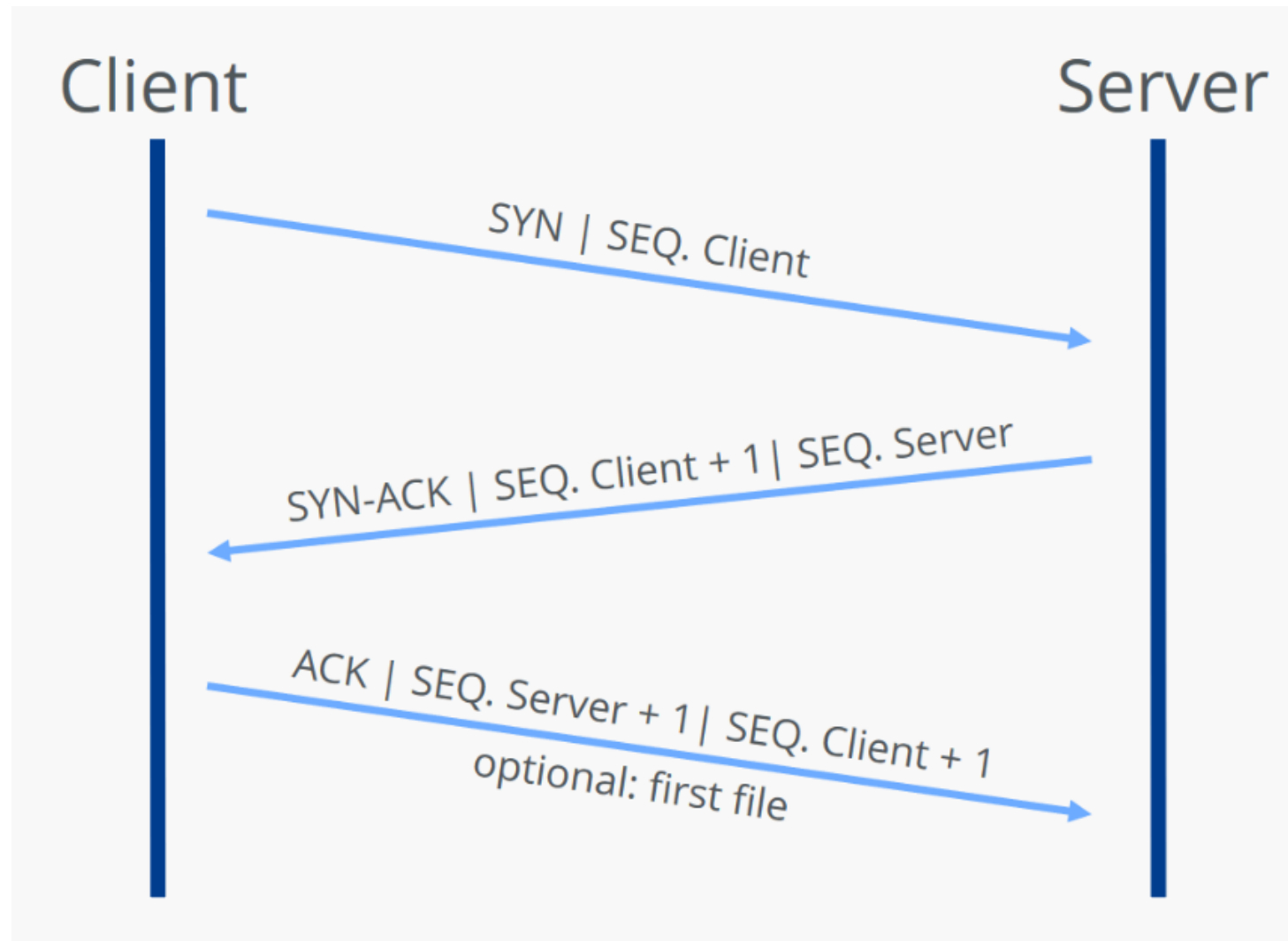
Su función principal es garantizar la transmisión confiable de datos entre dos dispositivos conectados en una red, generalmente sobre el protocolo IP (Internet Protocol).



Mientras que IP se encarga de direccionar y encaminar los paquetes, TCP asegura que los datos lleguen completos, en orden y sin errores



Orientado a conexión: Antes de transmitir datos, establece una conexión entre el cliente y el servidor mediante el proceso de three-way handshake.



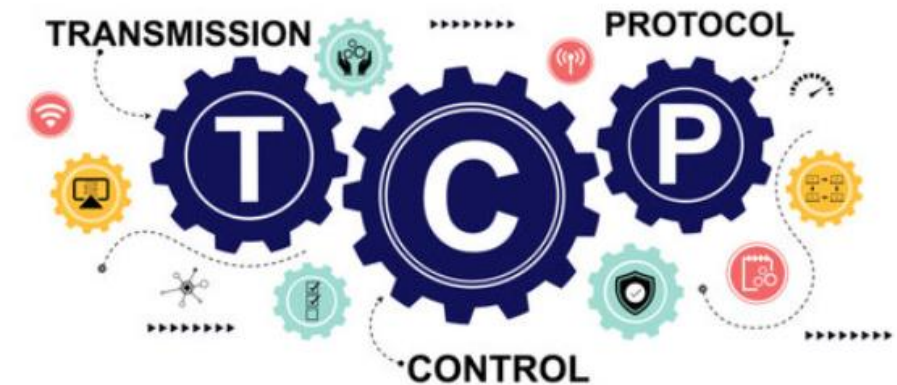
TCP 3-Way Handshake

Relación de TCP dentro del modelo OSI y TCP/IP.

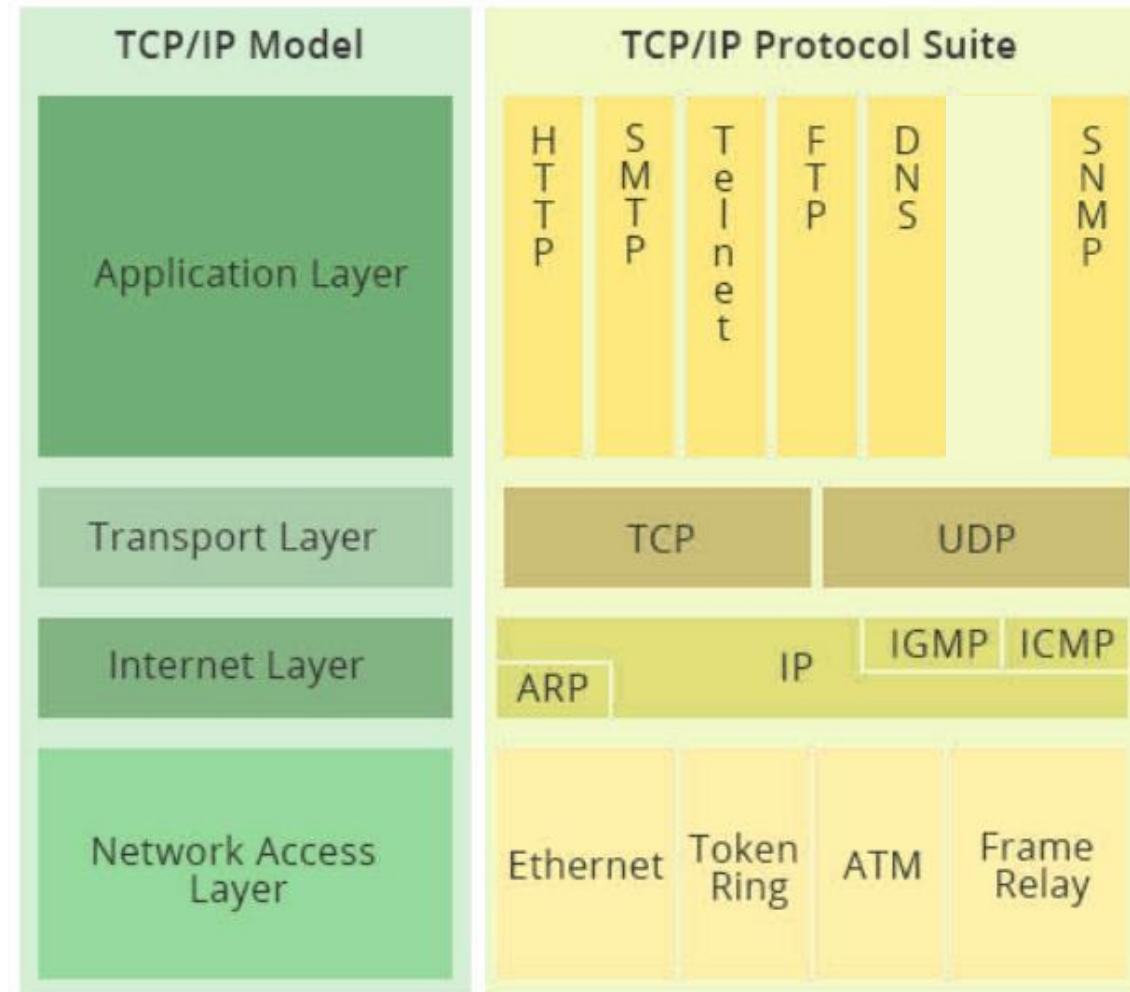
TCP se ubica en la **Capa de Transporte (capa 4)**.

Función en OSI:

- Proporciona **comunicación confiable** extremo a extremo.
- Divide los datos de las aplicaciones en segmentos.
- Realiza control de flujo, control de congestión y retransmisión.
- Se comunica directamente con:
 - La **Capa de Red (capa 3, IP)** para el envío de segmentos.
 - La **Capa de Sesión (capa 5)** y superiores, que le entregan los datos de las aplicaciones



Relación de TCP dentro del modelo OSI y TCP/IP.



Historia y evolución del protocolo TCP.

- En los años 70, con el desarrollo de ARPANET (precursor de Internet), se necesitaba un protocolo que permitiera la comunicación confiable entre computadoras heterogéneas.
- En 1973, Vinton Cerf y Robert Kahn propusieron el diseño inicial de un protocolo que combinaba lo que hoy conocemos como TCP/IP en un solo protocolo.
- 1974: Se publica el primer documento formal sobre Transmission Control Program, que integraba funciones de transporte y direccionamiento en un único protocolo.
- 1978: Se decide separar las funciones en dos protocolos distintos:
 - TCP (Transmission Control Protocol): Para la transmisión confiable de datos.
 - IP (Internet Protocol): Para el direccionamiento y envío de paquetes.
 - Nace así el modelo TCP/IP que conocemos.

Historia y evolución del protocolo TCP.



- 1981: Publicación del estándar RFC 793, que describe TCP de manera oficial.
- 1983: TCP/IP se adopta como el protocolo estándar en ARPANET, reemplazando a NCP (Network Control Protocol).
- Desde entonces, TCP se convierte en la base de Internet.
- TCP nació en los 70 como parte de un solo protocolo junto con IP, y fue evolucionando hasta convertirse en el estándar de transporte confiable de Internet. Hoy sigue siendo fundamental, pero con constantes mejoras (MPTCP, TFO) y con nuevos competidores que buscan optimizar velocidad y seguridad.

Arquitectura TCP

Establecimiento de conexión

Segmentación y encapsulamiento

Control de flujo y congestión

Fiabilidad

Cierre de conexión

Arquitectura TCP

1. Establecimiento de conexión

- A. Usa el three-way handshake (SYN, SYN-ACK, ACK) para iniciar la comunicación.
- B. Garantiza que ambos extremos estén listos para enviar y recibir datos

2. Segmentación y encapsulamiento

- A. TCP divide los datos de las aplicaciones en segmentos.
- B. Cada segmento incluye un número de secuencia, acknowledgment (ACK) y cabecera de control
- C. Los segmentos se entregan a la capa IP para ser enviados en paquetes.

3. Control de flujo y congestión

- A. Utiliza mecanismos como ventana deslizante (sliding window) para no saturar al receptor.
- B. Implementa algoritmos como Slow Start, Congestion Avoidance y Fast Retransmit para evitar congestiones en la red.

4. Fiabilidad

- A. Reenvía paquetes perdidos.
- B. Ordena los segmentos que llegan desordenados.
- C. Elimina duplicados.

5. Cierre de conexión

- A. Utiliza el four-way handshake con paquetes FIN y ACK para terminar de forma ordenada la sesión.

Arquitectura TCP

1. Control de flujo y congestión

Slow Start

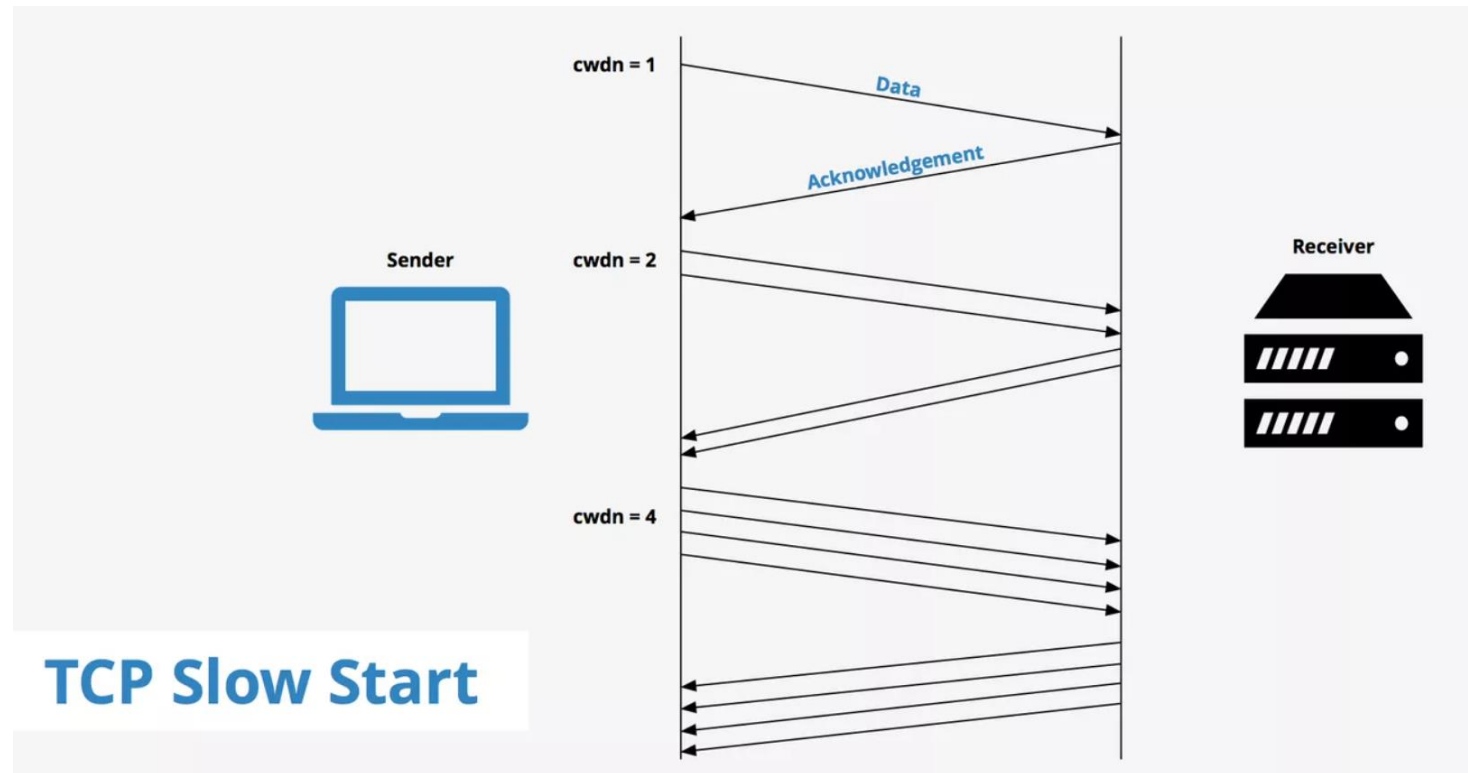
El Slow Start en TCP es un mecanismo de control de congestión que regula la cantidad de datos que un emisor puede enviar sin recibir aún confirmaciones (ACKs), con el objetivo de evitar la sobrecarga repentina en la red.

- Cuando una conexión TCP comienza, el emisor no conoce la capacidad real de la red (ancho de banda disponible).
- Para no saturarla de inmediato, inicia con una ventana de congestión (cwnd) pequeña, usualmente de 1 MSS (Maximum Segment Size).
- Cada vez que se recibe un ACK confirmando la entrega de datos, el tamaño de la ventana de congestión (cwnd) se incrementa en 1 MSS por cada ACK recibido.
- En la práctica, esto significa que por cada ronda de RTT (Round Trip Time), la cwnd se duplica.
- Ejemplo:
 - Inicio: cwnd = 1 MSS
 - Tras un RTT: cwnd = 2 MSS
 - Tras otro RTT: cwnd = 4 MSS
 - Luego: 8 MSS, 16 MSS, etc.

Arquitectura TCP

1. Control de flujo y congestión

Slow Start



Arquitectura TCP

1. Control de flujo y congestión

Congestion Avoidance

Es un mecanismo de control de congestión diseñado para crecer de manera más lenta y controlada la ventana de congestión (cwnd) una vez que TCP detecta que la red se acerca a su capacidad máxima.

Detección de congestión:

Si hay pérdida de paquetes (por timeout o triple ACK duplicado), TCP interpreta que ocurrió congestión.

Entonces:

- Ajusta $ssthresh = cwnd / 2$.
- Reduce cwnd (generalmente a 1 MSS en caso de timeout, o a ssthresh si fue triple ACK).
- Vuelve a iniciar Slow Start o permanece en Congestion Avoidance dependiendo del caso.

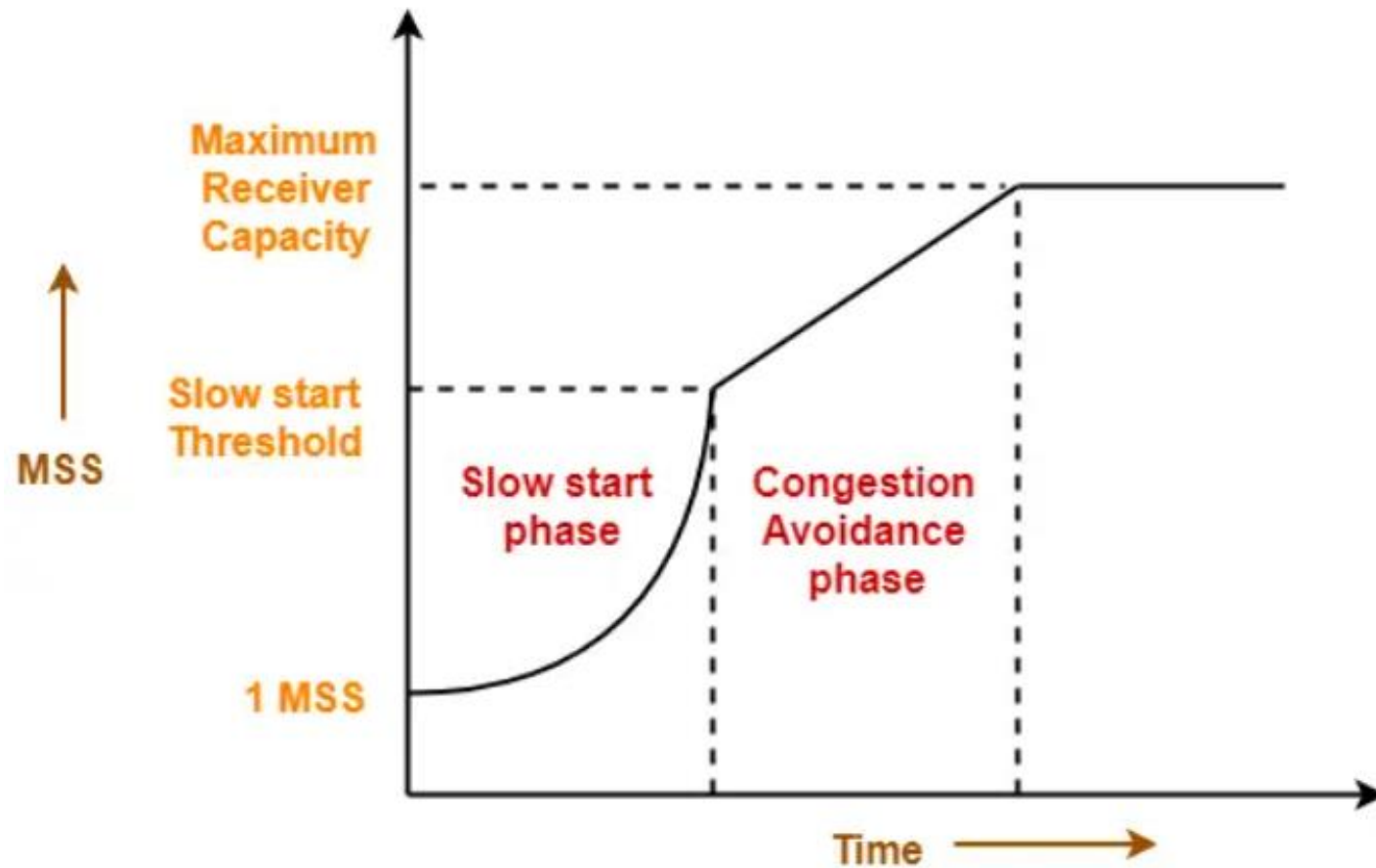
Ejemplo:

- Si $ssthresh = 16$, a partir de ahí entra en Congestion Avoidance
- $cwnd = 17, 18, 19, 20$ MSS... (lineal).

Arquitectura TCP

1. Control de flujo y congestión

Congestion Avoidance



Arquitectura TCP

1. Concepto de Comunicación Extremo a Extremo

El principio de comunicación extremo a extremo (end-to-end) es uno de los fundamentos de TCP.

Esto significa que:

El control de la comunicación ocurre únicamente entre los dispositivos finales (cliente y servidor).

- Ejemplo: Una computadora que envía un correo electrónico y el servidor que lo recibe.
- Los routers y dispositivos intermedios solo se encargan de encaminar paquetes; no verifican ni corrigen errores de transmisión.

Responsabilidad en los extremos:

- TCP en el emisor asegura que los datos salgan correctamente segmentados y numerados.
- TCP en el receptor se encarga de reensamblarlos, verificarlos y enviar confirmaciones (ACK)



Arquitectura TCP

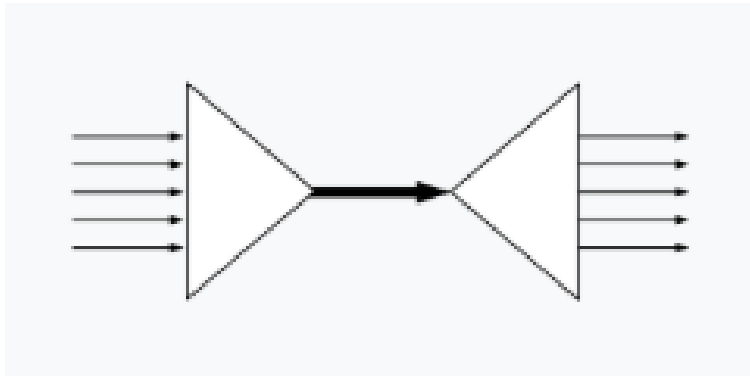
TCP	UDP
De confianza	Inconfiable
orientado a conexion	sin conexion
Retransmision de segmentos y control a traves de ventanas	No hay ventanas ni retransmisiones
Con secuencia	Sin secuencia
segmentos reconocidos	no reconocidos

Comparación entre TCP y UDP

Arquitectura TCP

Multiplexación TCP

La multiplexación en TCP significa que un mismo dispositivo (host) puede manejar múltiples conexiones simultáneas utilizando diferentes aplicaciones y servicios.)



Cómo lo logra TCP:

Usa el par de puertos (puerto de origen + puerto de destino) junto con las direcciones IP de origen y destino para identificar de manera única cada conexión.

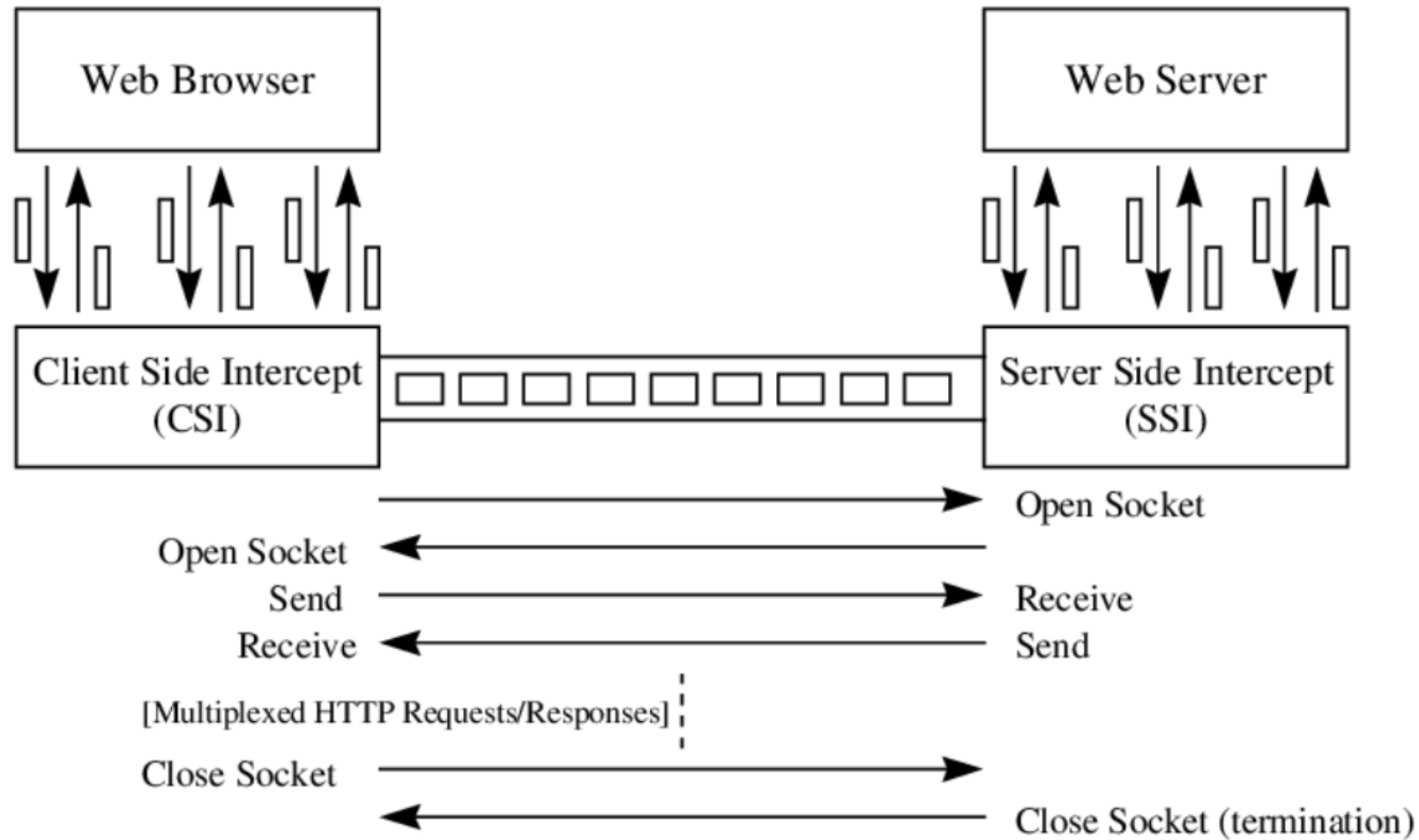
Esta combinación se llama socket o cuádrupla TCP

Ejemplo:

Tu computadora puede estar:

- Navegando una página web (HTTP 80 o HTTPS 443).
- Recibiendo correos (IMAP 143 o 993).
- Usando una conexión SSH (22).

Arquitectura TCP



Ejemplo de multiplexación en TCP

Casos de uso de TCP

HTTP / HTTPS (Navegación Web)

Puerto por defecto: 80 (HTTP), 443 (HTTPS con TLS/SSL).

Uso de TCP:

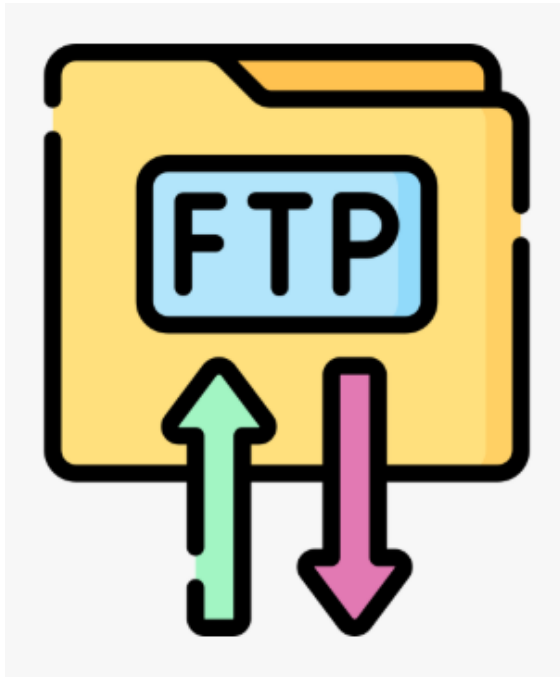
- Establece conexiones confiables entre navegadores y servidores web.
- Asegura que las páginas web (HTML, CSS, imágenes, scripts) se reciban completas y en orden.
- Con **HTTPS**, TCP se combina con TLS para cifrar los datos y garantizar seguridad.

Ejemplo:

Al abrir una página en tu navegador, TCP garantiza que cada fragmento del sitio llegue íntegro antes de renderizarse.



Casos de uso de TCP



FTP (File Transfer Protocol)

Puerto por defecto: 20 (datos) y 21 (control).

Uso de TCP:

Permite transferir archivos grandes sin errores.
Controla la retransmisión si un segmento se pierde.
Usa dos conexiones TCP: una para control (comandos) y otra para datos (archivos).

Ejemplo:

Al subir un archivo a un servidor FTP, TCP asegura que el archivo llegue completo, byte por byte, sin corrupción.

Flujo de datos en TCP



TCP recibe los datos de la aplicación.



Los divide en segmentos.



Añade cabecera con: puertos de origen/destino, número de secuencia, ACK, control de flujo, etc.

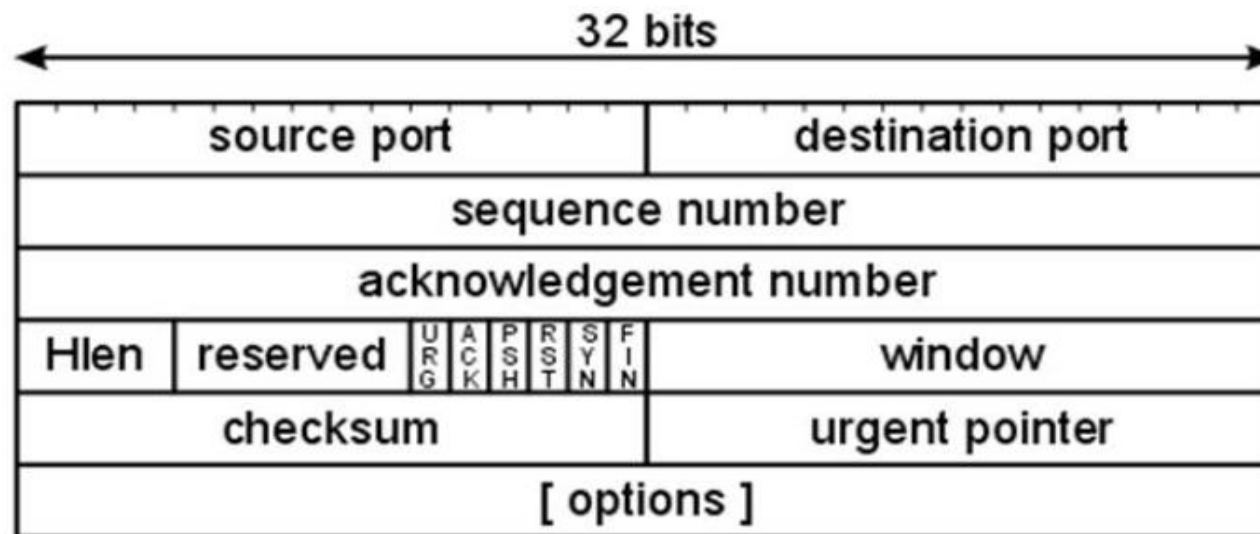


Se entrega a la capa de Internet.

Estructura del segmento TCP

El encabezado TCP es parte del segmento TCP y contiene la información de control necesaria para garantizar una comunicación confiable extremo a extremo.

Tiene un tamaño mínimo de 20 bytes y máximo de 60 bytes (dependiendo de las opciones).



TCP Header

Estructura del segmento TCP

Reservado (6 bits)

- Se reserva para usos futuros (normalmente en 0).

Banderas de Control (6 bits principales, aunque puede haber más según RFCs)

- URG: Datos urgentes.
- ACK: Indica que el campo de acuse de recibo es válido.
- PSH: Solicita que los datos se envíen inmediatamente a la aplicación.
- RST: Restablece la conexión.
- SYN: Inicia la conexión.
- FIN: Finaliza la conexión.

Tamaño de Ventana (16 bits)

- Control de flujo: indica cuántos bytes puede recibir el host sin saturarse.

Checksum (16 bits)

- Verifica la integridad del encabezado y de los datos TCP.

Puntero Urgente (16 bits): Indica el final de los datos urgentes, si la bandera URG está activa.

Estructura del segmento TCP

Puerto de Origen (16 bits)

- Identifica la aplicación emisora en el host de origen.

Puerto de Destino (16 bits)

- Identifica la aplicación receptora en el host destino.

Número de Secuencia (32 bits)

- Marca el orden de los bytes enviados.
- Permite que el receptor reensamble los segmentos en el orden correcto.

Número de Acuse de Recibo (ACK) (32 bits)

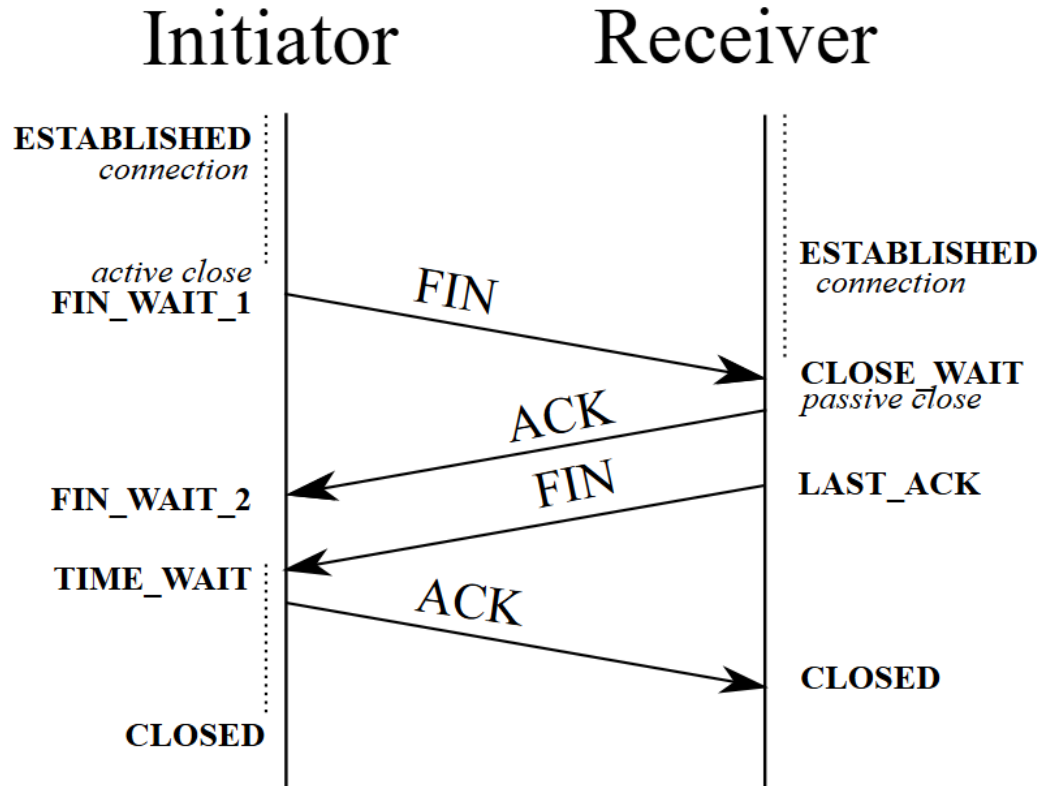
- Indica el próximo número de secuencia que el receptor espera.
- Permite confirmar la recepción de datos.

Longitud de Cabecera (4 bits)

- Indica dónde empiezan los datos dentro del segmento.
- También llamado Data Offset.



Establecimiento y cierre de conexión



Concepto de *handshake*

Definición:

El handshake TCP (también llamado three-way handshake) es el proceso de negociación inicial entre dos hosts que desean comunicarse mediante TCP.

Su propósito es:

- Establecer una conexión confiable.
- Sincronizar números de secuencia (para ordenar los datos).
- Confirmar que ambos extremos están listos para transmitir y recibir.

Establecimiento y cierre de conexión

Funcionamiento del Three-Way Handshake

Se utilizan tres pasos con banderas SYN y ACK en los segmentos TCP:

SYN (synchronize)

- El cliente envía un segmento con la bandera SYN=1 al servidor.
- Propone un número de secuencia inicial (ISN, Initial Sequence Number).

SYN-ACK (synchronize + acknowledgment)

- El servidor responde con un segmento que tiene SYN=1 y ACK=1.
- Acepta el número de secuencia del cliente con un ACK.
- Envía su propio número de secuencia inicial.

ACK (acknowledgment)

- El cliente envía un segmento con ACK=1, confirmando la recepción del número de secuencia del servidor.
- Ahora la conexión está establecida y ambos pueden comenzar a intercambiar datos.

Establecimiento y cierre de conexión

Importancia del Handshake TCP en Seguridad y Sincronización

El three-way handshake es esencial porque:

- Asegura la sincronización de números de secuencia:
 - Cada extremo propone un ISN (Initial Sequence Number) único.
 - Esto permite que los datos se reensamblen en el orden correcto.
- Confirma que ambos extremos están listos:
 - Cliente y servidor verifican que tienen capacidad para enviar/recibir datos antes de comenzar la transmisión.
- Evita pérdida de información:
 - Si un extremo no responde al SYN o al ACK, la conexión no se establece.



Ejemplo de captura con Wireshark del three-way handshake.

ip.addr == 65.61.137.117						
No.	Time	Source	Destination	Protoc	Leng	Info
3	0.066074746	10.0.2.151	65.61.137.117	TCP	74	58806 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2474736972
4	0.206960957	65.61.137.117	10.0.2.151	TCP	60	80 → 58806 [SYN, ACK] Seq=0 Ack=1 Win=32768 Len=0 MSS=1460
5	0.207035788	10.0.2.151	65.61.137.117	TCP	54	58806 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
35	1.474219554	10.0.2.151	65.61.137.117	HTTP	487	GET / HTTP/1.1

Ejemplo con protocolo HTTP

ip.addr == 10.0.2.153						
No.	Time	Source	Destination	Protoc	Leng	Info
5	1.605389677	10.0.2.151	10.0.2.153	TCP	74	40468 → 21 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=1139904232 TSecr=4294967295
6	1.606105597	10.0.2.153	10.0.2.151	TCP	74	21 → 40468 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM TSval=1139904232 TSecr=4294967295
7	1.606161519	10.0.2.151	10.0.2.153	TCP	66	40468 → 21 [ACK] Seq=1 Ack=1 Win=129940 Len=0 TSval=1139904232 TSecr=4294967295
8	1.615905984	10.0.2.153	10.0.2.151	FTP	86	Response: 220 (vsFTPd 2.3.4)

Ejemplo con protocolo FTP

Establecimiento y cierre de conexión

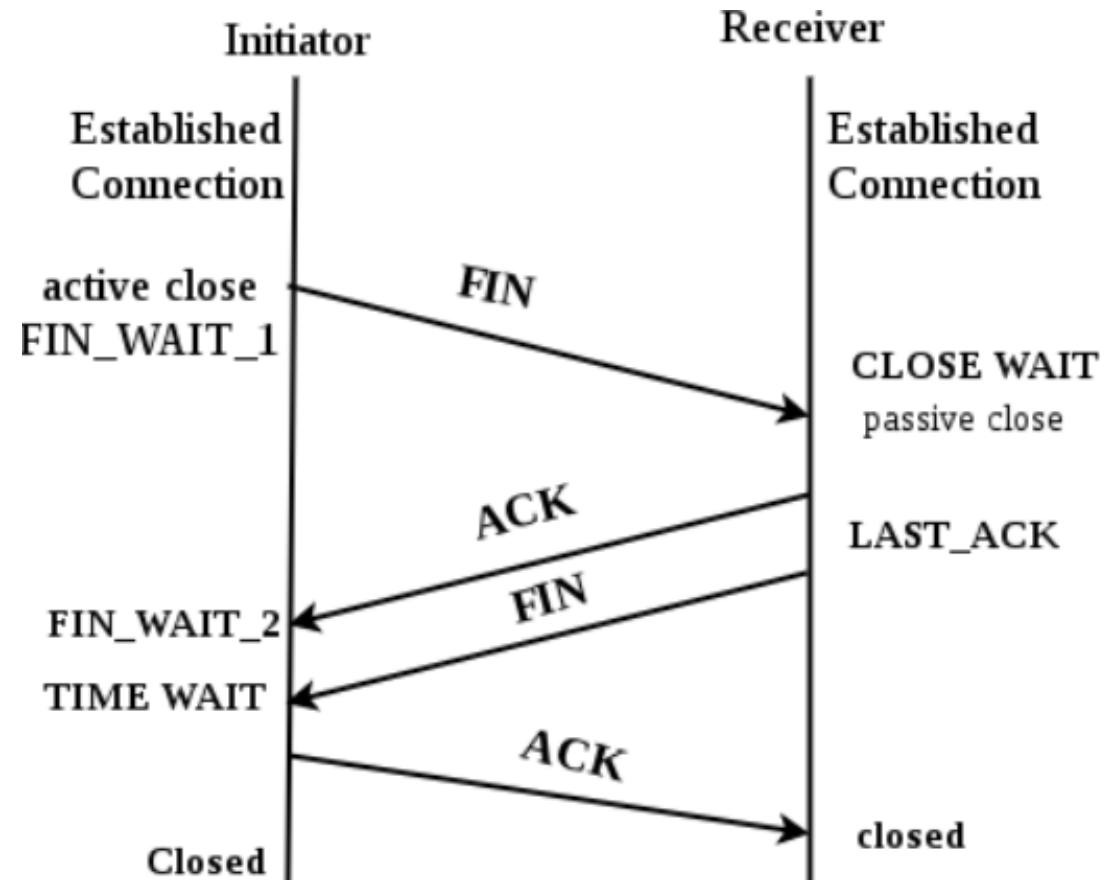
Proceso de cierre de conexión TCP (Four-Way Termination)

El proceso de cierre de conexión en TCP, conocido como Four-Way Termination (terminación en cuatro pasos), es el mecanismo mediante el cual dos dispositivos que estaban comunicándose finalizan de manera ordenada una conexión.

- FIN (Finalización de envío de datos): El host que quiere terminar la conexión (ejemplo: el Cliente A) envía un segmento FIN al otro host (Servidor B).
- ACK (Reconocimiento de FIN): El Servidor B recibe el FIN y responde con un ACK para confirmar que ha recibido la solicitud de cierre.
- FIN del segundo host: Cuando el Servidor B ya no tiene más datos que enviar, envía su propio FIN a Cliente A.
- ACK final: El Cliente A responde con un ACK, confirmando la recepción del FIN del Servidor B.
 - En este momento, la conexión se cierra completamente.

Establecimiento y cierre de conexión

Proceso de cierre de conexión TCP (Four-Way Termination)



Ejemplo de captura con Wireshark

Proceso de cierre de conexión (Four-Way Termination).

30	117.138427527	10.0.2.13	65.61.137.117	TCP	54	35086 → 80	[FIN, ACK]	Seq=1 Ack=1 Win=64240 Len=0
31	117.139149951	65.61.137.117	10.0.2.13	TCP	60	80 → 35086	[ACK]	Seq=1 Ack=2 Win=32767 Len=0
32	117.303579534	65.61.137.117	10.0.2.13	TCP	60	80 → 35086	[FIN, ACK]	Seq=1 Ack=2 Win=32767 Len=0
33	117.303611366	10.0.2.13	65.61.137.117	TCP	54	35086 → 80	[ACK]	Seq=2 Ack=2 Win=64240 Len=0

Ejemplo con protocolo HTTP

87	15.096196892	10.0.2.13	10.0.2.23	TCP	66	40448 → 21	[ACK]	Seq=49 Ack=198 Win=129756 Len=0 TSval=668073284 TSecr=66
88	15.096561200	10.0.2.23	10.0.2.13	TCP	66	21 → 40448	[FIN, ACK]	Seq=198 Ack=49 Win=5792 Len=0 TSval=913662 TSecr=66
89	15.096792297	10.0.2.13	10.0.2.23	TCP	66	40448 → 21	[FIN, ACK]	Seq=49 Ack=199 Win=129756 Len=0 TSval=668073284 TSecr=66
90	15.097693102	10.0.2.23	10.0.2.13	TCP	66	21 → 40448	[ACK]	Seq=199 Ack=50 Win=5792 Len=0 TSval=913662 TSecr=66

Ejemplo con protocolo FTP

Estados de conexión TCP

Los estados de conexión TCP son fundamentales para entender cómo funciona el protocolo en cada fase de la comunicación. TCP mantiene un estado de la conexión en cada extremo, que evoluciona conforme se intercambian los segmentos (SYN, ACK, FIN).

LISTEN: El servidor está esperando solicitudes de conexión entrantes en un puerto específico. Estado típico de un socket servidor antes de que ocurra el Three-Way Handshake.

ESTABLISHED: La conexión está activa y lista para enviar/recibir datos. Es el estado normal cuando dos extremos ya se comunican.

CLOSE-WAIT: El host recibió un FIN del otro extremo. Todavía puede enviar datos mientras no haya mandado su propio FIN. Espera a que la aplicación cierre la conexión.

TIME-WAIT: El host que envía el ACK final entra en este estado. Se mantiene un tiempo (normalmente $2 \times \text{MSL} = \text{Maximum Segment Lifetime}$) para asegurarse de que el último ACK no se pierda.

CLOSED: Estado final: no hay conexión activa ni pendiente. Puede ser inicial (antes de establecer la conexión) o final (después de cerrarla).

Estados de conexión TCP

```
└─# netstat -ant
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22               0.0.0.0:*               LISTEN
tcp        0      0 127.0.1.1:22            127.0.0.1:40764         ESTABLISHED
tcp        0      0 10.0.2.13:46534         34.107.243.93:443       ESTABLISHED
tcp        0      0 127.0.0.1:48008         127.0.1.1:22            ESTABLISHED
tcp        0      0 127.0.0.1:40764         127.0.1.1:22            ESTABLISHED
tcp        0      0 127.0.0.1:48258         127.0.0.1:22            TIME_WAIT
tcp        0      0 127.0.1.1:22            127.0.0.1:48008         ESTABLISHED
tcp6       0      0 :::80                   :::*                     LISTEN
tcp6       0      0 :::22                   :::*                     LISTEN
```

Ejemplo de estados de conexión en el protocolo TCP

Gestión de errores

Introducción

TCP ofrece entrega ordenada, sin pérdidas y sin duplicaciones de los datos. Para lograrlo, incorpora un conjunto de mecanismos de gestión de errores y medidas que contribuyen a la seguridad de la conexión.

La gestión de errores en TCP se centra en asegurar la integridad y consistencia de los datos transmitidos.

En paralelo, la seguridad en TCP aborda la necesidad de proteger la conexión frente a ataques y accesos no autorizados. Aunque TCP no fue concebido originalmente con fuertes mecanismos de seguridad, se han desarrollado extensiones y buenas prácticas.

En conjunto, estos mecanismos hacen que TCP sea un protocolo robusto, capaz de ofrecer no solo fiabilidad en la transmisión, sino también un nivel de protección ante fallos y amenazas comunes en redes abiertas como Internet.



Checksum TCP y Detección de Errores

Función del Checksum en TCP

El checksum de TCP es un valor numérico que se utiliza para detectar errores en los segmentos TCP durante la transmisión.

Se calcula a partir del contenido del encabezado TCP, los datos del segmento y una pseudo-cabecera extraída de IP.

Objetivos:

- Garantizar que el segmento recibido no haya sido alterado o dañado en tránsito debido a errores de transmisión (ruido, pérdidas de bits, corrupción de datos).
- Verificar la integridad de extremo a extremo, ya que se calcula en el host emisor y se valida en el host receptor.



Checksum TCP y Detección de Errores

Pseudo-cabecera para el Checksum

El cálculo del checksum TCP no se limita al encabezado TCP y a los datos. También incluye una pseudo-cabecera que refuerza la detección de errores, conformada por información crítica de la capa IP:

- Dirección IP de origen (32 bits)
- Dirección IP de destino (32 bits)
- Protocolo (8 bits → valor 6 para TCP)
- Longitud total del segmento TCP (16 bits)

Pseudo IP header (12 Bytes)

Source IP (32 bits)		
Destination IP (32 bits)		
Fixed 8 bits	Protocol Field (8 bits)	TCP segment length (16 bits)

Esta pseudo-cabecera asegura que el segmento llegue al destino correcto y ayuda a detectar problemas en la entrega a nivel de red.

Checksum TCP y Detección de Errores

Cálculo del Checksum

El proceso funciona de la siguiente manera:

El emisor realiza una suma complementada a uno (One's complement sum) de todos los campos:

- Encabezado TCP
- Datos del segmento
- Pseudo-cabecera IP

El resultado se coloca en el campo Checksum del encabezado TCP (16 bits).

En el receptor, se repite el mismo cálculo:

- Si el resultado final es todo "1" (valor esperado en complemento a uno), se considera que el segmento está intacto.
- Si no coincide, el segmento se descarta por error de transmisión.



Checksum TCP y Detección de Errores

Ejemplo

Parámetros usados:

- IP origen: 192.168.1.10 → bytes C0 A8 01 0A
- IP destino: 192.168.1.20 → bytes C0 A8 01 14
- Protocolo: 6 (TCP) → byte 06
- Longitud TCP (cabecera 20 + datos 4) = 24 → 0x0018 (00 18)
- Puerto origen: 12345 → 30 39 (0x3039)
- Puerto destino: 80 → 00 50 (0x0050)
- Seq: 00 00 00 01
- Ack: 00 00 00 00
- Offset/Flags: 50 18 (0x5018)
- Window: 40 00 (0x4000)
- Checksum (campo en cabecera, durante cálculo) = 00 00
- Urg pointer: 00 00
- Datos ASCII "ABCD" → 41 42 43 44

De ahí salen las **18 palabras de 16 bits** (en hex):

- 0xC0A8
- 0x010A
- 0xC0A8
- 0x0114
- 0x0006
- 0x0018
- 0x3039
- 0x0050
- 0x0000 (checksum campo = 0 durante cálculo)
- 0x0001 (parte baja del seq)
- 0x0000 (parte alta del ack)
- 0x0000
- 0x5018 (offset+flags)
- 0x4000 (window)
- 0x0000 (checksum, alto)
- 0x0000 (urg pointer)
- 0x4142 ('A'B')
- 0x4344 ('C'D')

Checksum TCP y Detección de Errores

Ejemplo

Regla: sumar palabra a palabra. Si la suma excede 0xFFFF, se toma el resultado & 0xFFFF y se añade 1 (end-around carry).

Paso 1: $0x0000 + 0xC0A8 = 0xC0A8$
Acumulado = $0xC0A8$

Paso 2: $0xC0A8 + 0x010A = 0xC1B2$
Acumulado = $0xC1B2$

Paso 3: $0xC1B2 + 0xC0A8 = 0x1825A$ (suma cruda)
 $0x1825A \& 0xFFFF = 0x825A \rightarrow + \text{carry } 1 = 0x825B$
Acumulado = $0x825B$

.....

Paso 18: $0x8572 + 0x4344 = 0xC8B6$
Acumulado final = $0xC8B6$

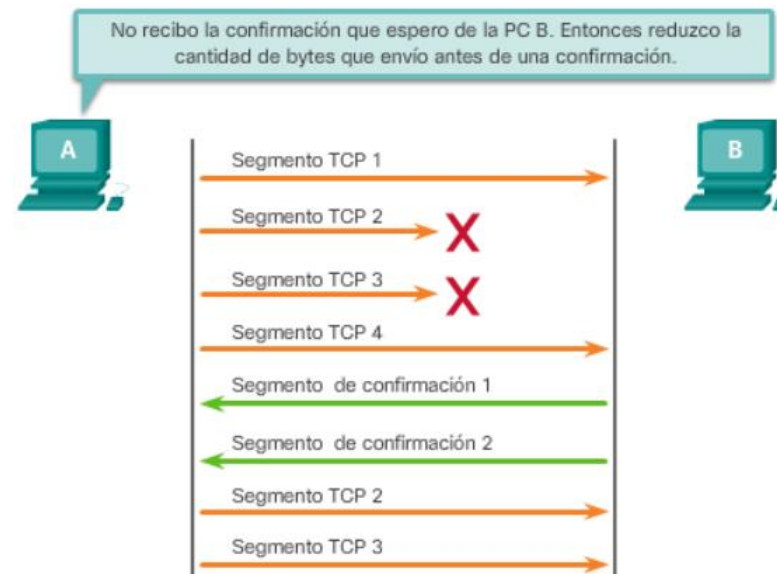


Retransmisión de segmentos perdidos en TCP

Una de las funciones clave de **TCP** es garantizar la **entrega confiable de datos** entre dos extremos de comunicación. Para lograrlo, TCP implementa un mecanismo de **detección y retransmisión** de segmentos perdidos.

¿Por qué se pierden los segmentos?

- Congestión en la red (paquetes descartados por routers).
- Errores en la transmisión física (interferencias, cortes).
- Desbordamiento de buffers en los dispositivos intermedios



Retransmisión de segmentos perdidos en TCP

Mecanismos de detección de pérdida

TCP usa dos enfoques principales:

a) Temporizador de retransmisión (RTO – Retransmission Timeout)

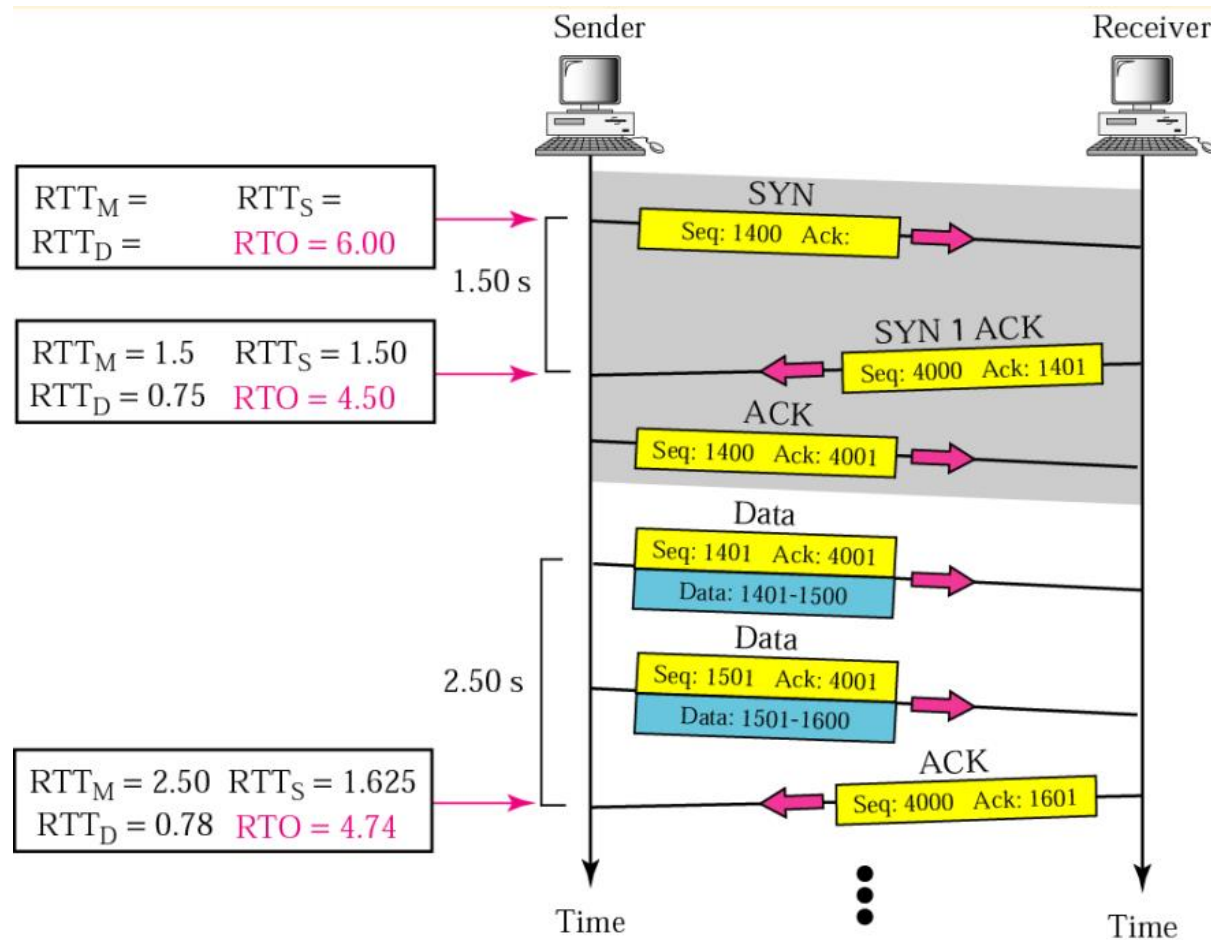
- Cada segmento transmitido inicia un temporizador.
- Si el **ACK** no llega antes de que el temporizador expire → el segmento se retransmite.
- El RTO se ajusta dinámicamente con el **RTT (Round Trip Time)** estimado.

b) Retransmisión rápida (Fast Retransmit)

- Si un emisor recibe **tres ACK duplicados** (el receptor confirma repetidamente el mismo número de secuencia), interpreta que un segmento intermedio se perdió.
- TCP retransmite de inmediato el segmento faltante, sin esperar el RTO.

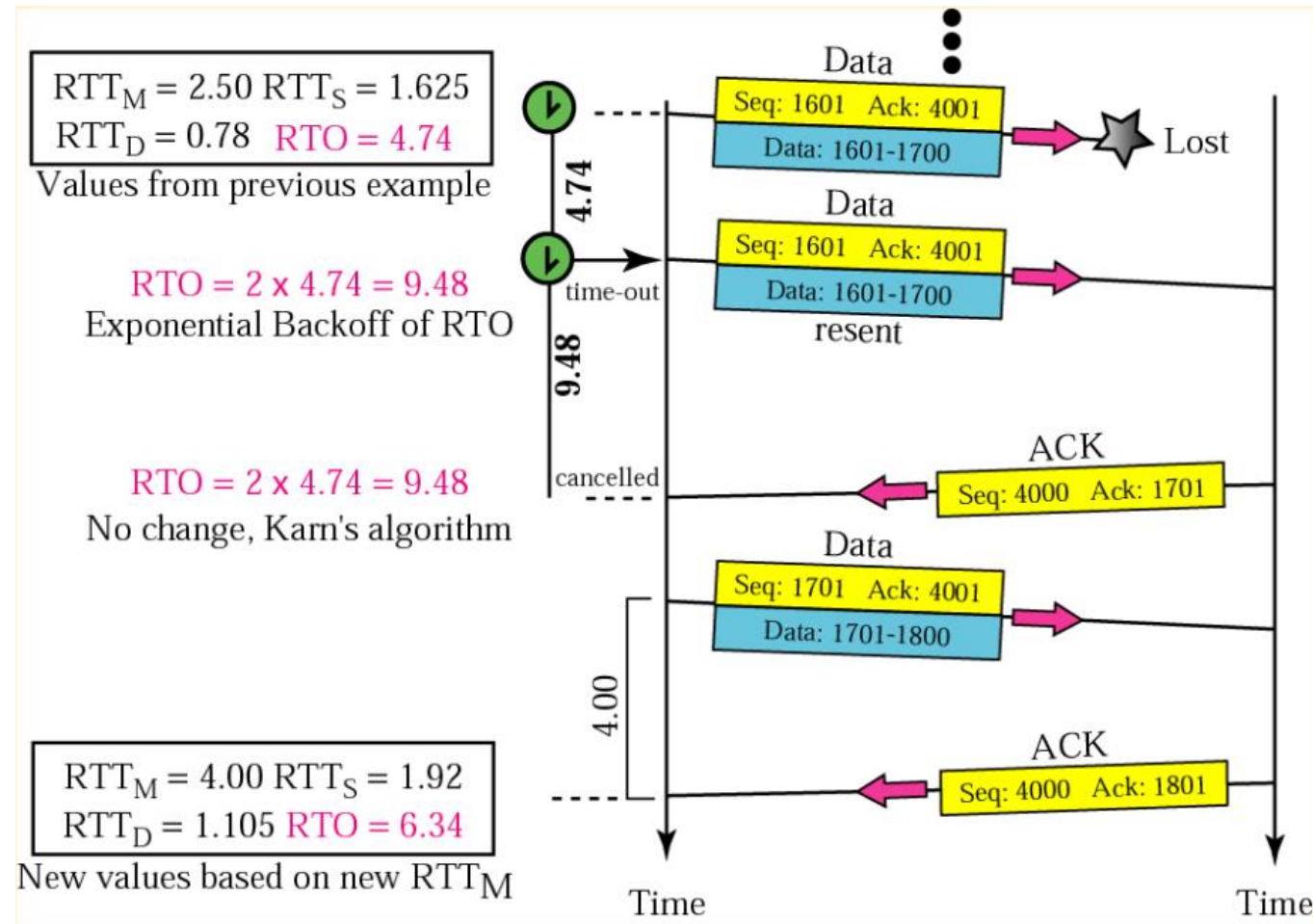
TCP retransmite segmentos perdidos mediante timeouts (RTO) y retransmisión rápida con ACK duplicados. Con SACK, mejora aún más la eficiencia al identificar qué datos faltan sin reenviar todo.

Retransmisión de segmentos perdidos en TCP



Temporizador de retransmisión

Retransmisión de segmentos perdidos en TCP



Retransmisión rápida

Control de flujo en TCP

El control de flujo en TCP es un mecanismo que evita que un emisor sature al receptor enviándole más datos de los que puede procesar o almacenar en su buffer de recepción. Su objetivo es mantener el equilibrio entre emisor y receptor para garantizar comunicación eficiente y confiable.

El problema:

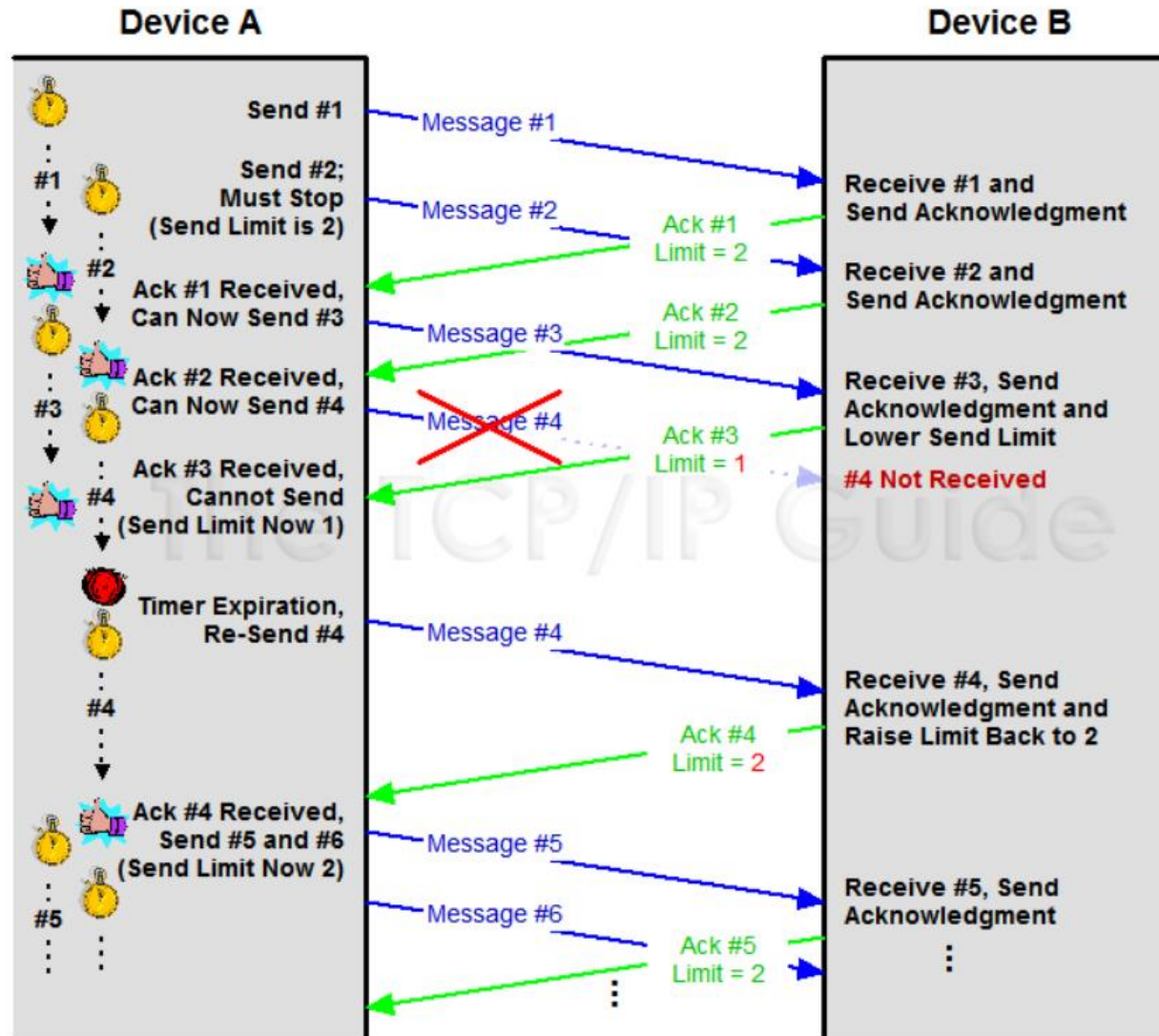
Los dispositivos en una conexión TCP no siempre tienen la misma capacidad de procesamiento o almacenamiento. Si el emisor envía datos muy rápido, el receptor podría desbordar su buffer, causando pérdidas. Se necesita una forma de regular la velocidad de envío.

La solución: Ventana deslizante (Sliding Window)

- TCP implementa control de flujo mediante el **mecanismo de ventana deslizante**:
- El receptor anuncia en cada **ACK** el tamaño de su **ventana de recepción (rwnd, receive window)**.
- El emisor solo puede enviar datos dentro de ese rango de bytes permitido.
- A medida que el receptor procesa y libera espacio en su buffer, amplía la ventana y lo comunica al emisor.

Esto asegura que el emisor nunca envíe más datos de los que el receptor puede manejar.

Control de flujo en TCP



**Ventana deslizante
(Sliding Window)**

Cierre

El **Transmission Control Protocol (TCP)** es el pilar fundamental de las comunicaciones confiables en Internet. A lo largo de su funcionamiento, TCP asegura que los datos viajen **de extremo a extremo** de manera íntegra, ordenada y libre de pérdidas, adaptándose a las condiciones de los sistemas y de la red.

Sus principales aportes son:

- **Confiabilidad:** garantiza la entrega de los datos mediante confirmaciones (ACK) y retransmisiones.
- **Orden correcto:** utiliza números de secuencia para reconstruir los segmentos en el orden adecuado.
- **Integridad:** el checksum detecta errores en los segmentos.
- **Control de flujo:** regula el envío para no saturar al receptor.
- **Control de congestión:** ajusta dinámicamente la transmisión para evitar sobrecargar la red.
- **Conexiones orientadas:** establece y finaliza sesiones mediante el handshake de tres vías y la terminación en cuatro pasos.

En conclusión, TCP es el equilibrio entre confiabilidad y eficiencia, y su diseño ha permitido que servicios críticos como web, correo electrónico, transferencia de archivos y aplicaciones seguras funcionen correctamente a nivel global.